

Experiment No. 14

Title

Alpha-Beta Pruning Algorithm for Gaming Using Artificial Intelligence

Aim

To develop a Python program to implement the Alpha-Beta Pruning algorithm for game-playing applications.

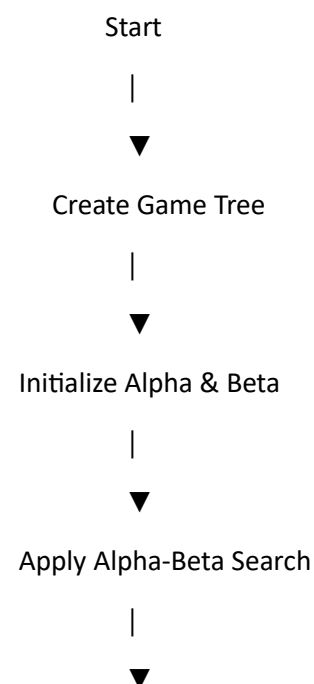
Objective

The objective of this experiment is to understand the Alpha-Beta Pruning algorithm and implement it to improve the efficiency of the Minimax algorithm by eliminating unnecessary branches.

Algorithm

- Step 1:** Create the game tree with utility values.
 - Step 2:** Initialize Alpha and Beta values.
 - Step 3:** Apply the Minimax algorithm recursively.
 - Step 4:** Update Alpha and Beta values at each node.
 - Step 5:** Prune branches when $\text{Alpha} \geq \text{Beta}$.
 - Step 6:** Continue until the root node is evaluated.
 - Step 7:** Display the optimal value.
-

Flowchart



Update Alpha/Beta

|



Alpha $\geq$ Beta ?

|

|

Yes

No

|

|

Prune Branch Continue Search

|

|



|



Display Best Move

|



Stop

Python Program

MAX = 1000

MIN = -1000

```
def alphabeta(depth, node, maximizing, values, alpha, beta):
```

```
    if depth == 3:
```

```
        return values[node]
```

```
    if maximizing:
```

```
        best = MIN
```

```
    for i in range(2):
```

```
        value = alphabeta(depth + 1, node * 2 + i,
```

```

        False, values, alpha, beta)

    best = max(best, value)
    alpha = max(alpha, best)

    if beta <= alpha:
        break

    return best

else:
    best = MAX

    for i in range(2):
        value = alphabeta(depth + 1, node * 2 + i,
                           True, values, alpha, beta)
        best = min(best, value)
        beta = min(beta, best)

    if beta <= alpha:
        break

    return best

values = [3, 5, 6, 9, 1, 2, 0, -1]

result = alphabeta(0, 0, True, values, MIN, MAX)

print("Optimal Value:", result)

```

Sample Output

Optimal Value: 5

Result

The optimal game value was successfully obtained using the Alpha-Beta Pruning algorithm.

Conclusion

Thus, the Alpha-Beta Pruning algorithm was successfully implemented. It reduces the number of nodes evaluated by the Minimax algorithm, improving efficiency while still producing the optimal decision. It is widely used in game-playing AI such as Chess, Checkers, and Tic Tac Toe.